

LangChain Agents — Análisis completo de la Sesión 12

Fuente base: *Agentes IA — LangChain Agents* — Módulo 5 (Herramientas para Orquestación), Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado. Dictada por MEng. Boris Alzamora.

Complementado con: investigación propia sobre el paper original de **ReAct** (Yao et al., 2022), la documentación oficial de LangChain 1.x (`create_agent`), el concepto de **Agent Harness** ("arnés del agente") tal como lo describen actualmente LangChain (DeepAgents) y otros autores de la comunidad, y la comparación entre LangChain, LangGraph y LangSmith como piezas de un mismo ecosistema.

Propósito de este documento: esta sesión abre el **Módulo 5 (Herramientas para Orquestación)**, dejando atrás el marco puramente conceptual de los Módulos 3 y 4 para entrar al *framework* concreto — LangChain — con el que el estudiante va a implementar sus agentes de aquí en adelante. El eje de la sesión es doble: (1) **planificación** (*planning*) como capacidad explícita del agente, con **ReAct** como técnica bisagra entre razonamiento y acción; y (2) la primera implementación formal de un **Tool Calling Agent** en LangChain.

0. Dónde se ubica esta sesión — de la teoría del agente a su implementación

Módulo 3 – Ingeniería de Prompts:

Zero-shot, Few-shot, CoT, ReAct, modularización de prompts

└ Técnicas de prompting en abstracto, sin atarlas a un framework.

Módulo 4 – Agentes Cognitivos (Sesiones 8-11):

Arquitectura → Memoria → Tipos reflexivos → Colaboración

└ Vocabulario y diseño conceptual de agentes, agnóstico de herramienta.

Módulo 5 – Herramientas para Orquestación (arranca en esta Sesión 12):

LangChain / LangGraph / LangSmith

└ Se aterriza todo lo anterior en un framework concreto: cómo se construye un agente que planifica y llama herramientas, en código.

La sesión abre con un *recap* explícito de la Sesión 11 (página 2 del PDF, un simple rótulo "Recap Ses11" sin contenido propio en la diapositiva — el repaso se hace verbalmente en clase) y luego encadena dos bloques temáticos: **Planning con Prompt Engineering (ReAct/CoT)** primero, y **LangChain Agents** después.

1. Objetivos y agenda de la sesión

Objetivos declarados en el PDF (página 4):

1. Comprender la importancia del *Planning* en agentes.
2. Aprender a implementar un *LangChain agent* de herramientas.

Referencia oficial citada en el propio PDF:

<https://python.langchain.com/docs/tutorials/agents/>

Agenda del Bloque A — Planning y ecosistema (página 5):

#	Tema
1	Prompt Engineering — Planning, ReAct
1.1	Reflexión: Herramientas y Roles (entorno)
2	LangChain Ecosystem
3	LangChain Agents (con introducción a <i>Agent Harness</i>)
4	Demo: LangChain Agents desde el código

Agenda del Bloque B — Implementación (página 9):

#	Tema
1	LangChain Agents invocando herramientas
2	Demo
3	Lab: Implementar un Agente de Herramientas de LangChain en clase

Tarea final indicada en la página 17 del PDF:

Tarea — Agente ReAct Personal. Asuma una necesidad personal de su vida diaria e implemente un agente ReAct usando LangChain. Individual (**Personal**). Entrega: Google Doc con **[objetivo, código]**. **Due Date: 29/07**. Nota adicional en la diapositiva: **[AI TRANSPARENCY POLICY] -10**.

Nota sobre la política de transparencia de IA: el PDF incluye literalmente esa anotación de "-10" junto a la política de transparencia de IA, sin más detalle en la diapositiva. La lectura razonable es que el curso penaliza con 10 puntos el no declarar transparentemente

el uso de IA en la entrega (p. ej., no aclarar qué partes del código o del objetivo se generaron con asistencia de un modelo). Conviene confirmar el detalle exacto de esta política con el docente antes de la entrega, ya que el PDF no desarrolla la rúbrica completa.

2. Planning — por qué esta sesión empieza aquí

2.1 Las ocho técnicas de *Prompt Engineering* (página 6)

El PDF resume el mapa completo de técnicas de *prompting* visto en el Módulo 3, y lo reordena en tres niveles de madurez — la lectura implícita es que **Planning** (vía ReAct) es la culminación de ese mapa, no una técnica aislada:

Nivel	#	Técnica	Nombre completo / qué hace
Básico	1	<i>Zero-Shot Prompting</i>	Sin ejemplos entregados al modelo
Básico	2	<i>Few-Shot Prompting</i>	2–5 ejemplos entregados para respuestas más específicas
Básico	3	<i>Chain-of-Thought (CoT)</i>	Razonamiento paso a paso para tareas complejas
Intermedio	4	<i>Self-Consistency</i>	Genera múltiples rutas de razonamiento y agrega (promedia/vota) los resultados
Intermedio	5	<i>Tree of Thoughts (ToT)</i>	Explora múltiples rutas de decisión simultáneamente
Avanzado	6	RAG (<i>Retrieval Augmented Generation</i> , "Generación Aumentada por Recuperación")	Añade información externa recuperada de una base de datos/documentos al prompt
Avanzado	7	ART (<i>Automatic Reasoning and Tool-use</i> , "Razonamiento Automático y Uso de Herramientas")	El propio modelo decide cuándo y cómo invocar herramientas externas como parte del razonamiento
Avanzado	8	ReAct Prompting (<i>Reasoning and Acting</i> , "Razonamiento y Actuación")	Combina razonamiento y acción en un mismo ciclo iterativo

Esta tabla no es un simple repaso: **RAG**, **ART** y **ReAct** son, en esencia, las tres técnicas de las que un *Tool Calling Agent* (el tema del resto de la sesión) toma prestado algo — RAG para recuperar contexto externo, ART/ReAct para decidir cuándo invocar una herramienta y razonar sobre su resultado.

2.2 Ejercicio de reflexión — cuatro prompts, un mismo objetivo (página 7)

El PDF presenta cuatro variantes del mismo *prompt* ("Escribe un párrafo emocionante del caballero Carmelo"), en un orden deliberadamente progresivo que ilustra en la práctica qué agrega cada nivel de sofisticación:

Prompt 1 — Zero-shot puro:

```
Escribe un parrafo emocionante del caballero carmelo
```

Prompt 2 — Few-shot (un ejemplo de referencia):

```
Escribe un parrafo emocionante del caballero carmelo

Para ello, toma como ejemplo la Batalla contra aji seco.
```

Prompt 3 — Chain-of-Thought (análisis antes de escribir):

```
Escribe un parrafo emocionante del caballero carmelo

Para ello, analiza que conoces sobre el caballero carmelo,
plantea un parrafo emocionante y escríbelo
```

Prompt 4 — Planning explícito con herramientas + Reflexión (el más completo):

```
Dada la tarea de escribir un parrafo emocionante del caballero carmelo,
elabora un plan considerando las herramientas de analista, escritor y
critico. El plan debe ser definido por steps y en este formato:
{step:, task:, tool_name:, instructions:, parameters:[]}. Y tomaras la
respuesta de una herramienta como parte de invocacion de la siguiente.
Una vez elaborado el plan ejecutalo conforme tus herramientas y entrega
una finalmente los comentarios del critico
```

Y, en paralelo, una variante que añade explícitamente el paso de **autocrítica** (*self-refine*) sobre el resultado de la técnica CoT:

```
Escribe un parrafo emocionante del caballero carmelo

Para ello, analiza que conoces sobre el caballero carmelo, plantea un
parrafo emocionante y escríbelo
```

Analiza el parrafo que creaste, haz una autocritica y conforme a ello actualiza el parrafo.

Por qué importa este ejercicio: el Prompt 4 es, en miniatura, la definición operativa de **Planning** que la sesión va a formalizar en la Sección 3: el modelo no responde directamente, sino que primero produce un **plan estructurado** (una lista de `steps`, cada uno con una herramienta (`tool_name`), instrucciones y parámetros), y luego **ejecuta ese plan encadenando resultados** de una herramienta como entrada de la siguiente. Esto es exactamente el patrón que después implementa código (`create_agent` con *tools*), solo que aquí se pide simular el rol de "herramienta" (analista, escritor, crítico) dentro del propio *prompt*, sin código real detrás. El añadido de autocrítica, por su parte, es una instancia manual y simplificada del patrón **Reflexion/Self-Refine** que ya se había investigado en la Sesión 10 (generar → criticar → refinar).

2.3 Lab — Actualizar el *System Prompt* del proyecto propio (página 8)

Actualice el *system prompt* que tiene hoy de su proyecto hacia un patrón **ReAct** o **CoT** que nos ayude a evidenciar aspectos de **planificación** o **enrutamiento de tareas**. — Grupal, con Copilot Studio, ChatGPT u otra herramienta. **10 min.**

Este *lab* es el puente directo entre el ejercicio teórico de la Sección 2.2 y el proyecto final del estudiante: no se trata de escribir un *prompt* nuevo desde cero, sino de **auditar y reescribir** el *system prompt* que el proyecto ya tiene, para que se note explícitamente cuándo el agente planifica (decide una secuencia de pasos) o enruta (decide a qué herramienta/rol delegar cada paso), en vez de responder todo de un tirón.

3. LangChain Agents — definición formal (página 10)

Definición: un agente es un sistema que utiliza un **gran modelo de lenguaje** (LLM, *Large Language Model*) como motor de razonamiento para determinar la secuencia de acciones a realizar para alcanzar un objetivo.

Diferencia clave con las "Cadenas" (*Chains*):

	<i>Chains</i> (cadenas)	<i>Agents</i> (agentes)
Flujo de acciones	Predeterminado y fijo, definido por el desarrollador	Dinámico: el LLM decide en tiempo real qué herramientas usar y en qué orden

	Chains (cadenas)	Agents (agentes)
Ciclo de trabajo	Ejecución lineal de pasos ya codificados	Ciclo de acción → observación → razonamiento , repetido hasta encontrar la respuesta final

Esta distinción retoma, con otro vocabulario, algo que ya se había discutido en la Sesión 7 del Módulo 3 (*Workflow vs. Agent*): una *Chain* es un flujo agéntico controlado (*workflow*), mientras que un *Agent* tiene autonomía real sobre la secuencia de pasos — la diferencia entre "yo (desarrollador) decido el orden" y "el LLM decide el orden".

3.1 El ciclo de trabajo, formalizado

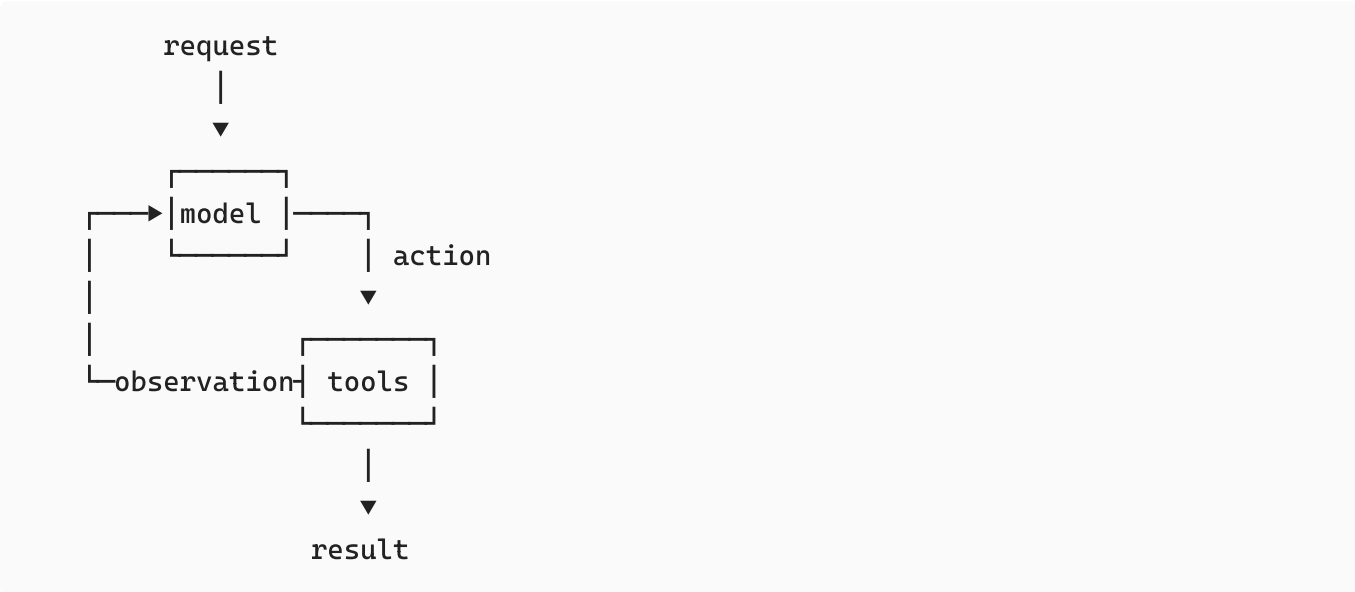
El propio PDF resalta en cursiva y subrayado la frase "**acción-observación-razonamiento**". Usando la notación de agentes ya introducida en la Sesión 10 (percepción → estado → acción), el ciclo de un *LangChain Agent* puede escribirse como:

$$a_t = \pi_{\text{LLM}}(h_t), \quad o_t = \text{tool}(a_t), \quad h_{t+1} = h_t \oplus (a_t, o_t)$$

donde h_t es el historial acumulado de razonamientos, acciones y observaciones hasta el paso t ; π_{LLM} es la política del modelo (qué acción tomar dado el historial); y el ciclo se repite hasta que el LLM decide que ya tiene la respuesta final, en vez de invocar una herramienta más.

3.2 Diagrama del bucle agente-herramienta (página 11)

"Un agente es un modelo que llama a herramientas en un bucle hasta que se completa una tarea determinada."



Este diagrama es la versión mínima y genérica del ciclo ReAct: `request` entra al `model`; el modelo decide una `action` (llamar una tool); la tool devuelve una `observation`; esa

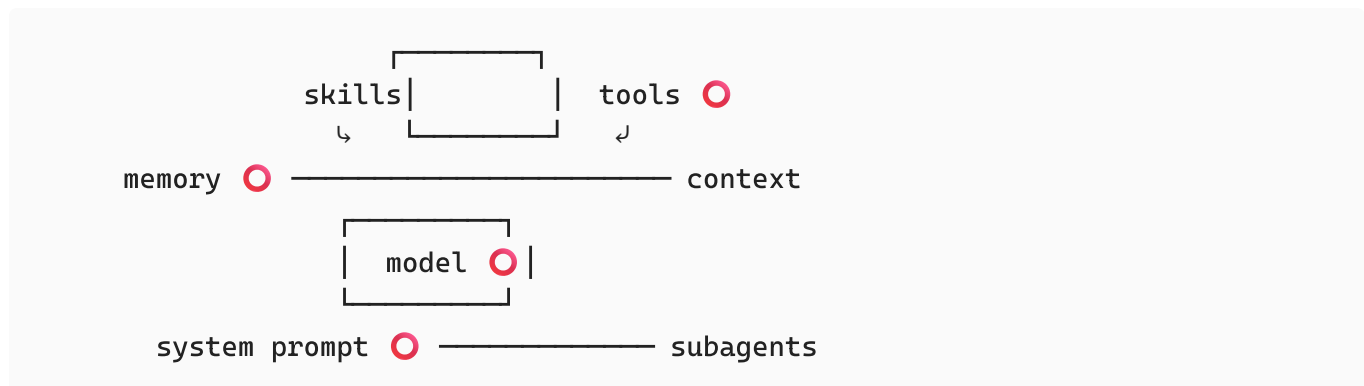
observación regresa al `model`, que decide si necesita otra acción o si ya puede entregar el `result` final.

3.3 Diagrama — "agent = model + harness" (página 11)

El PDF presenta una segunda pieza visual, más rica, con la ecuación:

```
agent = model + harness
```

Alrededor del `model` (en el centro), el diagrama dispone seis piezas conectadas por líneas punteadas — **memory**, **system prompt**, **tools**, **context**, **subagents** y **skills** —, de las cuales **memory**, **system prompt**, **tools** y **model** aparecen resaltadas con un círculo rojo en la diapositiva:



Lectura de este diagrama: el **modelo** por sí solo (los pesos del LLM) no es el agente. El agente es el modelo **más** todo lo que lo rodea y lo hace operar sobre un objetivo — a ese "todo lo demás" se le llama **harness** (literalmente, "arnés"): el andamiaje de código, configuración y lógica de ejecución que no es el modelo en sí. Los cuatro elementos resaltados en rojo (*memory*, *tools*, *model*, *system prompt*) son precisamente los que la sesión va a implementar en código con `create_agent` más adelante; *skills*, *context* y *subagents* quedan anunciados pero no se desarrollan aún — son terreno de sesiones futuras (ver Sección 3.4).

Relación con la Sesión 8: este diagrama es, en esencia, una reformulación moderna de la arquitectura de agente que ya se había presentado en la Sesión 8 (comunicación + contexto + entorno + autonomía + criticidad). *System prompt* ≈ *Context Definition*; *tools* ≈ *Environment Definition* (la parte de herramientas); *memory* ≈ *Short/Long Term Memory* de la Sesión 9; *subagents* ≈ el patrón de colaboración multiagente de la Sesión 11. Lo nuevo es el término **harness**, que agrupa a todos esos componentes bajo un solo nombre y los distingue explícitamente del modelo.

3.4 La pieza que queda para más adelante — *Harness Engineering* (página 12)

Inmediatamente después del diagrama anterior, el PDF inserta una imagen de transición — un caballo estilizado con arnés, en clave visual futurista — con el rótulo:

HARNESS ENGINEERING

Sesión 23 – Módulo 8

Es una diapositiva puente, no contenido de esta sesión: anuncia que el concepto de "harness" introducido aquí de forma superficial (Sección 3.3) será retomado y profundizado formalmente en la **Sesión 23 del Módulo 8**, bajo el nombre de *Harness Engineering*. Vale la pena que el estudiante guarde el diagrama de la Sección 3.3 como referencia, ya que esa sesión futura presumiblemente construirá sobre él.

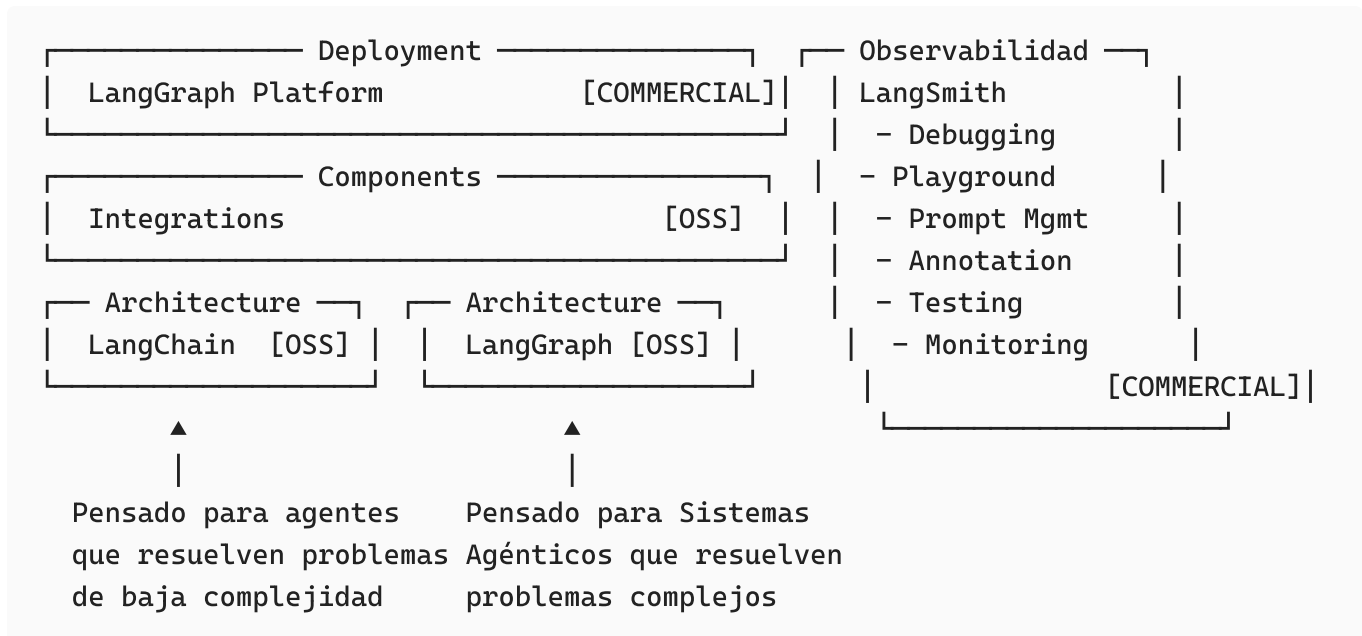
4. El ecosistema LangChain (páginas 13-14)

4.1 Tres características clave (página 13)

#	Característica	Explicación del PDF	Pieza del ecosistema
1	Interfaces estandarizadas para componentes — <i>"One Chain to ruled them all!"</i>	Hoy hay tantos modelos y herramientas para IA que cada uno viene con su propia API (<i>Application Programming Interface</i>), lo cual complica aprenderlas todas, combinarlas o cambiar de proveedor. LangChain ofrece una interfaz común.	LangChain
2	Orquestación	A medida que las aplicaciones combinan más modelos y herramientas, se necesita una forma eficiente de conectarlos y coordinarlos en un flujo de trabajo ordenado.	LangChain + LangGraph
3	Observabilidad y evaluación	Cuando las aplicaciones crecen en complejidad, entender qué pasa dentro de ellas se vuelve difícil; además, con tantas opciones de modelos/prompts, los desarrolladores pueden "paralizarse" al decidir el mejor equilibrio entre precisión, velocidad y costo.	LangSmith

4.2 Arquitectura del ecosistema (página 14)

"LangChain pasó de ser framework a ser ahora un ecosistema basado en diversos paquetes."



Expansión de siglas: **OSS** = *Open Source Software* (software de código abierto, sin costo de licencia); *Commercial* = capa de pago (hospedaje, colaboración en equipo, SLAs).

Capa	Componente	Licencia	Rol
Architecture	LangChain	OSS	Framework base: interfaces estandarizadas, <code>create_agent</code> , <i>tools</i> . Pensado para agentes que resuelven problemas de baja complejidad .
Architecture	LangGraph	OSS	Motor de orquestación por grafos de estado (nodos, <i>edges</i> , <i>checkpointers</i> , <i>store</i> — ya usado en las Sesiones 9 y 11). Pensado para Sistemas Agénticos que resuelven problemas complejos .
Components	Integrations	OSS	Conectores hacia proveedores de modelos, bases de datos vectoriales, APIs externas, etc.
Deployment	LangGraph Platform	Comercial	Despliegue gestionado de grafos LangGraph en producción.
Observabilidad	LangSmith	Comercial	<i>Debugging</i> , <i>Playground</i> , gestión de <i>prompts</i> , anotación, <i>testing</i> y monitoreo — la plataforma que se configura en detalle en <code>CONECTAR_LANGSMITH.md</code> (ver Sección 6).

Lectura práctica para el proyecto del estudiante: esta sesión usa `create_agent` de LangChain (arquitectura de baja complejidad) precisamente porque el caso de uso — un agente de herramientas para un cálculo presupuestal — no necesita el control fino de un grafo LangGraph explícito. Las Sesiones 9 y 11 sí usaron LangGraph directamente (`StateGraph` , `SqliteSaver` , `InMemoryStore`) porque necesitaban controlar memoria persistente y estrategias de gestión de mensajes a mano. La elección de herramienta, otra vez, depende del nivel de complejidad real del problema — el mismo principio de "usa el nivel mínimo que resuelve el problema" que ya había aparecido en la Sesión 10 para los tipos de agente reflexivo.

5. Demo y Laboratorio — Tool Calling Agent

5.1 Demo (página 15)

Tool Calling Agent using Langchain and Ollama

La demo en clase usa **Ollama** (modelo local, sin costo de API) para mostrar en vivo cómo un agente de LangChain decide invocar una herramienta.

5.2 Enunciado del laboratorio (página 16)

Implemente un *tool calling agent* usando LangChain que realice el cálculo presupuestal de materiales de su oficina.

5.3 Implementación real — `agente_presupuesto_materiales.py`

Archivo de la carpeta de la sesión, construido con `langchain.agents.create_agent` — la misma **API** usada en `agente_analista_reclamos.py` de la Sesión 11. El propio archivo se documenta como continuación directa de ese patrón.

Las tres *tools* del agente, en cadena obligatoria:

Orden	Tool	Qué hace	Por qué
1	<code>consultar_precio_material(nombre_material)</code>	Busca el precio unitario y la unidad de medida de un material en un catálogo interno (<code>CATALOGO_MATERIALES</code> , un diccionario Python con 21 artículos de oficina)	El LLM solo lee el catálogo "alucinaciones"
2	<code>calcular_subtotal_item(precio_unitario, cantidad)</code>	Multiplica <code>precio_unitario × cantidad</code>	Un LLM puede hacer aritmética simple
3	<code>generar_presupuesto_final(items, incluir_igv)</code>	Recibe la lista completa de ítems ya cotizados (validada con un modelo Pydantic <code>ItemPresupuesto</code>), recalcula los subtotales desde cero (nunca confía en los números que "recuerda" el LLM), suma el subtotal general, aplica IGV (<i>Impuesto General a las Ventas</i> , 18% en Perú) si corresponde, y guarda el reporte en Markdown	Repite el proceso con confianza por el agente en el agente de la

Búsqueda tolerante a errores de escritura:

```
def _normalizar(texto: str) -> str:
    """Quita tildes y pasa a minúsculas..."""

    sugerencias = get_close_matches(
        _normalizar(nombre_material), CATALOGO_MATERIALES.keys(), n=3, cutoff=0.5
    )
```

Si el material no existe en el catálogo, la *tool* usa `difflib.get_close_matches` para sugerir las entradas más parecidas (p. ej., si el usuario escribe "lapicero azul", puede sugerir "lapicero"), en vez de simplemente fallar o dejar que el LLM invente un precio.

Modelo Pydantic para el ítem cotizado:

```
class ItemPresupuesto(BaseModel):
    material: str
    cantidad: float
    precio_unitario: float
    subtotal: float
```

Este *schema* estructurado es lo que permite a `generar_presupuesto_final` recibir `items: list[ItemPresupuesto]` con garantía de tipos, en vez de un texto libre que habría que volver a parsear (a diferencia del formato de bloques de texto plano usado en la Sesión 11 para los reclamos bancarios — aquí, al ser todo un solo proceso con `create_agent`, se puede usar directamente *structured output* vía Pydantic).

Flujo obligatorio codificado en el *system prompt*:

1. `consultar_precio_material` — para CADA material, antes de calcular nada.
2. `calcular_subtotal_item` — nunca multiplicar "a mano".
3. `generar_presupuesto_final` — exactamente una vez, con la lista completa.
4. Responder con un resumen breve: ítems, subtotal, IGV y total.

Selección de proveedor de modelo en tiempo de ejecución:

```
LLM_PROVIDER = os.getenv("LLM_PROVIDER", "anthropic").lower()

if LLM_PROVIDER == "ollama":
    MODEL_ID = f"ollama:{os.getenv('OLLAMA_MODEL', 'llama3.2')}"
else:
    MODEL_ID = f"anthropic:{os.getenv('ANTHROPIC_MODEL', 'claude-sonnet-4-6')}"
```

Esto reproduce, en un solo archivo, la misma flexibilidad que `main.py` de la Sesión 11 lograba con una función `construir_modelo()`: el mismo agente puede correr sobre **Anthropic** (de pago, vía API) o sobre **Ollama** (modelo local, gratuito) sin tocar el resto del código — solo cambiando la variable de entorno `LLM_PROVIDER`. `create_agent` resuelve el string `"proveedor:modelo"` automáticamente.

Sin memoria entre presupuestos: al igual que `agente_analista_reclamos.py` de la Sesión 11, cada solicitud de presupuesto es independiente; no hay *checkpointing* porque no hace falta recordar una conversación previa para calcular un presupuesto nuevo.

5.4 `prompt.txt` — un artefacto de prueba aparte

La carpeta de la sesión también contiene `prompt.txt` , un *system prompt* corto para un caso distinto (un agente de agendamiento de citas de una veterinaria, con horario de atención de 3pm a 7pm de lunes a viernes). No está conectado al código de `agente_presupuesto_materiales.py` ; parece un borrador o prueba independiente de *system prompt*, útil como ejemplo adicional de *prompt* corto y orientado a una sola tarea (agendar citas conociendo disponibilidad y datos del paciente), pero no forma parte del flujo del laboratorio documentado en el PDF.

6. Observabilidad con LangSmith — `CONECTAR_LANGSMITH.md`

La carpeta de la sesión incluye una guía propia, ya redactada, para conectar el agente de presupuesto a **LangSmith** (la pieza de observabilidad del ecosistema descrita en la Sección 4.2). Resumen de sus pasos:

Paso	Acción
1	Crear cuenta en <code>smith.langchain.com</code> y generar una API key (<code>lsv2_pt_...</code> o <code>lsv2_sk_...</code>) desde Settings → API Keys
2	Completar en <code>Sesion12/.env</code> : <code>LANGSMITH_TRACING=true</code> , <code>LANGSMITH_API_KEY=...</code> , <code>LANGSMITH_PROJECT=sesion12-presupuesto-materiales</code>
3	Ejecutar <code>agente_presupuesto_materiales.py</code> normalmente — no requiere ningún cambio de código, porque el script ya llama a <code>load_dotenv()</code>
4	Revisar en la web de LangSmith, por cada <code>agent.invoke(...)</code> : el árbol completo de la ejecución (mensaje → modelo → tool invocada → resultado → siguiente llamada), tokens de entrada/salida, costo estimado en USD (<i>United States Dollar</i>) y latencia por paso
5 (opcional)	Separar corridas de prueba de corridas "reales" cambiando <code>LANGSMITH_PROJECT</code> por línea de comandos, sin tocar el código

Detalle importante que la guía aclara: el costo mostrado en LangSmith es una **estimación** basada en tarifas públicas y conteo de tokens, no una factura real; y si se corre el agente con `LLM_PROVIDER=ollama` , LangSmith sigue trazando la ejecución, pero el costo estimado aparece en `$0.00` porque Ollama no cobra por token.

Esta guía conecta directamente con la característica #3 del ecosistema (Sección 4.1, "Observabilidad y evaluación"): es la aplicación práctica, sobre el propio laboratorio de la sesión, de lo que el PDF solo describe en abstracto.

7. Investigación complementaria

7.1 ReAct — el paper detrás de la técnica (Yao et al., 2022)

El término **ReAct** que aparece en la página 6 del PDF proviene del paper "*ReAct: Synergizing Reasoning and Acting in Language Models*" (Yao, Zhao, Yu, Du, Shafran, Narasimhan y Cao, 2022). Su idea central: los modelos de lenguaje mejoran en el uso de herramientas cuando **razonan en voz alta antes de cada acción**, no solo antes de la respuesta final. El ciclo que proponen es:

$$\text{Thought}_t \rightarrow \text{Action}_t \rightarrow \text{Observation}_t \rightarrow \text{Thought}_{t+1} \rightarrow \dots$$

En cada ciclo, el modelo produce un **Thought** (paso de razonamiento explícito que interpreta el estado actual y determina qué hacer), luego una **Action** (invocación de una herramienta que interactúa con el entorno), y el valor devuelto por la herramienta se vuelve la **Observation**, que se añade al contexto e informa el siguiente razonamiento. El paper demostró que ReAct superaba significativamente al *prompting* de Chain-of-Thought puro en tareas intensivas en conocimiento (HotpotQA, FEVER) y en tareas de toma de decisiones (ALFWorld, WebShop). El diagrama `request → model → tools → observation → result` de la página 11 del PDF (Sección 3.2 de este documento) es, en esencia, la misma estructura de ciclo con otro nombre de variables.

7.2 El término "*agent harness*" — de dónde viene y por qué importa

La ecuación `agent = model + harness` que aparece en la página 11 del PDF no es una invención del curso: refleja un consenso reciente de la propia comunidad LangChain (y de otros laboratorios, incluyendo Anthropic, que usa el mismo término para describir la capa de andamiaje alrededor de Claude en productos como Claude Code — el propio entorno donde se genera este documento es, en ese sentido, un ejemplo vivo de "harness"). La idea: el modelo por sí solo no decide *cómo* ejecutar una tarea de principio a fin — necesita un **arnés** que le entregue *tools*, memoria, un *system prompt*, contexto de ejecución y (opcionalmente) delegación a *subagentes*. Fuentes recientes de la comunidad describen un *harness* más completo (como *DeepAgents*, de LangChain) que añade sobre esa base: herramientas de planificación explícitas, un sistema de archivos virtual, delegación a subagentes, ingeniería de contexto, memoria persistente, *skills*, ejecución de código en sandbox y soporte de supervisión humana (**HITL**, *Human-in-the-loop*) — es decir, exactamente las piezas que el diagrama de la página 11 deja anunciadas sin desarrollar (*skills*, *context*, *subagents*) y que se prometen para la Sesión 23 (*Harness Engineering*, Sección 3.4).

7.3 LangChain 1.x `create_agent` — qué hay detrás de la abstracción

`create_agent` (usado tanto en esta sesión como en las Sesiones 9 y 11) no es una implementación paralela a LangGraph: está **construido sobre LangGraph**. Acepta los mismos

parámetros que un grafo compilado (`checkpointer=` , `store=`), pero expone un nivel de abstracción más alto: en vez de escribir un nodo `call_model` a mano (como sí hacía `main.py` de la Sesión 11), se declara un `system_prompt` fijo y una lista de `tools` , y LangChain arma internamente el bucle "modelo → decide llamar una tool → ejecuta la tool → vuelve al modelo" — el mismo ciclo ReAct de la Sección 7.1, ya empaquetado.

7.4 Comparación con otros *frameworks* de orquestación

Dado que el Módulo 5 se llama explícitamente "Herramientas para Orquestación", vale la pena situar a LangChain/LangGraph dentro del panorama más amplio de *frameworks* de agentes, aunque el PDF no los mencione:

Framework	Enfoque principal	Comparación con LangChain/LangGraph
AutoGen (Microsoft)	Conversaciones multiagente configurables por código	Más centrado en el patrón de "agentes que chatean entre sí"; LangGraph es más explícito en el control de estado como grafo
CrewAI	Roles y tareas de agentes con una API de alto nivel (<i>crews</i>)	Más opinado y rápido de prototipar; LangChain ofrece más control fino a cambio de más código
OpenAI Agents SDK	<i>Handoffs</i> entre agentes y <i>guardrails</i> nativos del proveedor	Atado al ecosistema de un solo proveedor de modelo; LangChain es agnóstico de proveedor (Anthropic, OpenAI, Ollama, Google, etc., como ya se vio en las Sesiones 9 y 11)
Semantic Kernel (Microsoft)	Orientado a integrar agentes en aplicaciones .NET/Python empresariales	Más enfocado en integración empresarial que en el ciclo de razonamiento del agente en sí

Ninguno de estos compite exactamente en el mismo eje: la elección depende de la pila tecnológica existente, el lenguaje del equipo y cuánto control fino se necesita sobre el flujo (ver también la comparación LangChain-vs-LangGraph de la Sección 4.2: "baja complejidad" vs. "sistemas agénticos complejos" es un eje que reaparece, con matices, en la comparación entre estos *frameworks*).

8. Conexión con las sesiones anteriores

Sesión	Concepto previo	Cómo se retoma en la Sesión 12
Sesión 7 (Módulo 3)	<i>Workflow</i> (flujo agéntico controlado) vs. <i>Agent</i> (autonomía real)	Es exactamente la distinción "Chains vs. Agents" de la Sección 3 de este documento, con nombres nuevos
Sesión 8	Comunicación + Contexto + Entorno + Autonomía + Criticidad	El diagrama "agent = model + harness" es una reformulación de esos mismos componentes bajo el término <i>harness</i>
Sesión 9	<i>Short/Long Term Memory, Store, Checkpointer</i>	El nodo "memory" del diagrama de <i>harness</i> ; y la elección de no usar <i>checkpointer</i> en el laboratorio de esta sesión reproduce la misma lógica de "solo usar la memoria que el problema realmente necesita"
Sesión 10	Tipos de agente reflexivo (Simple Reflex → Learning)	El agente de presupuesto es, en esos términos, un Model-Based/Goal-Based Reflex Agent : reúne datos (precio, cantidad) y aplica una secuencia fija de reglas (consultar → calcular → generar) hacia una meta clara (presupuesto correcto), sin aprendizaje ni utilidad ponderada
Sesión 11	Colaboración por archivos, <i>Agent-to-Agent</i>	El nodo "subagents" del diagrama de <i>harness</i> apunta hacia el mismo problema que resolvía A2A: coordinar más de un agente. Aquí queda solo anunciado, no implementado

9. Mapa del repositorio de la sesión

```

Sesion12_LangChain_Agents/
  SES12_M5_Langchain_Agents_Tools.pdf
  agente_presupuesto_materiales.py    ← Lab: Tool Calling Agent (create_agent +
3 tools + Pydantic)
  prompt.txt                          ← Borrador de system prompt aparte
(agendamiento veterinario)
  CONECTAR_LANGSMITH.md               ← Guía propia para activar
tracing/observabilidad
  .env                                ← Credenciales (ANTHROPIC_API_KEY,
variables de LangSmith)
  .gitignore                          ← Ignora .env, __pycache__/ y
presupuestos_generados/

```


9.1 Dependencias observadas en el código

```
langchain>=1.x          (create_agent, tool)
langchain-anthropic      (model="anthropic:claude-sonnet-4-6")
langchain-ollama         (model="ollama:llama3.2", alternativa local)
pydantic                 (ItemPresupuesto)
python-dotenv
langsmith                (ya viene como dependencia de langchain; solo
                           requiere variables de entorno)
```

10. Síntesis — qué se lleva el estudiante de esta sesión

1. **Planning no es una técnica más de la lista: es lo que distingue a un *Agent* de una *Chain*.** Una *Chain* ejecuta un flujo que el desarrollador ya decidió; un *Agent* usa al LLM para decidir, en tiempo real, la secuencia de acciones — y ReAct es la técnica de *prompting* que hace ese ciclo explícito (razonar → actuar → observar → razonar de nuevo).
2. `agent = model + harness` es el marco mental para entender por qué "solo el modelo" nunca es suficiente: *tools*, *memory* y *system prompt* (lo implementado hoy) más *skills*, *context* y *subagents* (lo anunciado para la Sesión 23) son parte constitutiva del agente, no accesorios opcionales.
3. **LangChain, LangGraph y LangSmith no son tres productos separados, son tres capas de un mismo ecosistema:** interfaz estandarizada + orquestación + observabilidad. Elegir entre `create_agent` (LangChain, baja complejidad) y un `StateGraph` manual (LangGraph, sistemas complejos) es una decisión de diseño, no una preferencia estética — el mismo principio de "usar el nivel mínimo necesario" que ya regía la elección entre tipos de agente reflexivo en la Sesión 10.
4. **El laboratorio de presupuesto de materiales es la plantilla a reutilizar en la tarea del 29/07:** catálogo/fuente de verdad fuera del LLM, cálculo numérico delegado a código determinista, y un *system prompt* que impone explícitamente el orden de invocación de herramientas — ese mismo esqueleto sirve para construir el Agente ReAct Personal que pide la tarea final.